

Inheritance

Inheritance is used to derive the current class from another base class.

Derive the current class from a base class using the extend key:

```
• Osp.Class.define("My.Great.SuperClass",
• {
•   // This is the base class
• });
• Osp.Class.define("My.Cool.Class",
• {
•   extend: My.Great.SuperClass
• });
```

Single and multi-level inheritance is supported. You can derive a class from a base class which in turn has been derived from another base class:

```
• Osp.Class.define("Osp.Test.Cat",
• {
•   extend: Osp.Test.Animal
• });
• Osp.Class.define("Osp.Test.Kitten",
• {
•   extend: Osp.Test.Cat
• });
```

Multiple inheritance (deriving a class from multiple base classes) is not supported in the web applications.

Mixins

Mixins are collections of code and variables in a generic form that can be merged ("mixed") into other classes. They are similar to classes, but cannot be instantiated. Use mixins to share functionalities without using inheritance, and to extend or patch the functionalities of existing classes.

Typically, mixins are made up of only a few members that allow generic implementation of some very specific functionality. The code in the mixin must be generic, since it is mixed into different existing classes. Mixins usually cover only a single functionality aspect, and therefore tend to be small.

Declaring mixins

Mixins are declared using the Mixin keyword with an uppercase 'M':

```
• Osp.Mixin.define(name, config)
```

A mixin declaration contains properties, constructor, destructor, and